Pontifícia Universidade Católica de Minas Gerais
Curso de Engenharia de Software · Projeto de Software
Profa. Milena Menezes Adão

Laboratório 3 – Sprint 2 (Lab03S02)

Sistema de Moeda Estudantil

Modelo ER, estratégia de acesso ao banco de dados (ORM + Repository/DAO)
e CRUDs de aluno e empresa parceira (versão inicial: front-end e back-end)

Integrante: Gustavo Lima Dias

Belo Horizonte – 2º semestre de 2026

1. O que esta sprint entrega

Esta entrega implementa a primeira versão funcional do sistema sobre a modelagem da Sprint 1, mantendo a arquitetura MVC definida no diagrama de componentes:

- **Modelo ER** com as 11 tabelas do sistema, incluindo as que só serão usadas na Sprint 3 (transações), para que o banco já nasça completo.
- **Estratégia de acesso a dados**: ORM com JPA/Hibernate e padrão Repository/DAO com Spring Data JPA.
- **CRUD de aluno** (cadastrar, listar, consultar, editar, excluir) e **CRUD de empresa parceira**, com o cadastro das vantagens que cada empresa oferece.
- **Front-end web** (HTML, CSS e JavaScript) que se comunica com o back-end por uma API REST em JSON.
- **Modelos UML atualizados** (v2 dos diagramas de classes e de componentes) para refletir o código.

Tecnologias

Camada	Tecnologia	Papel
View	HTML5, CSS3, JavaScript (fetch API)	Telas de listagem e formulários; consome a API REST
Controller	Spring Boot 3.3 · Spring Web	Endpoints REST, validação de entrada (Bean Validation), tratamento de erros
Model – Serviços	Spring (@Service, @Transactional)	Regras de negócio: CPF/CNPJ válidos e únicos, email e login únicos, senha
Model – Persistência	Spring Data JPA · Hibernate 6	Repositórios (DAO) e mapeamento objeto-relacional
Banco de dados	H2 (arquivo local) · PostgreSQL (perfil opcional)	Armazenamento relacional
Segurança	spring-security-crypto (BCrypt)	Hash das senhas; login completo na Sprint 3

2. Modelo Entidade-Relacionamento

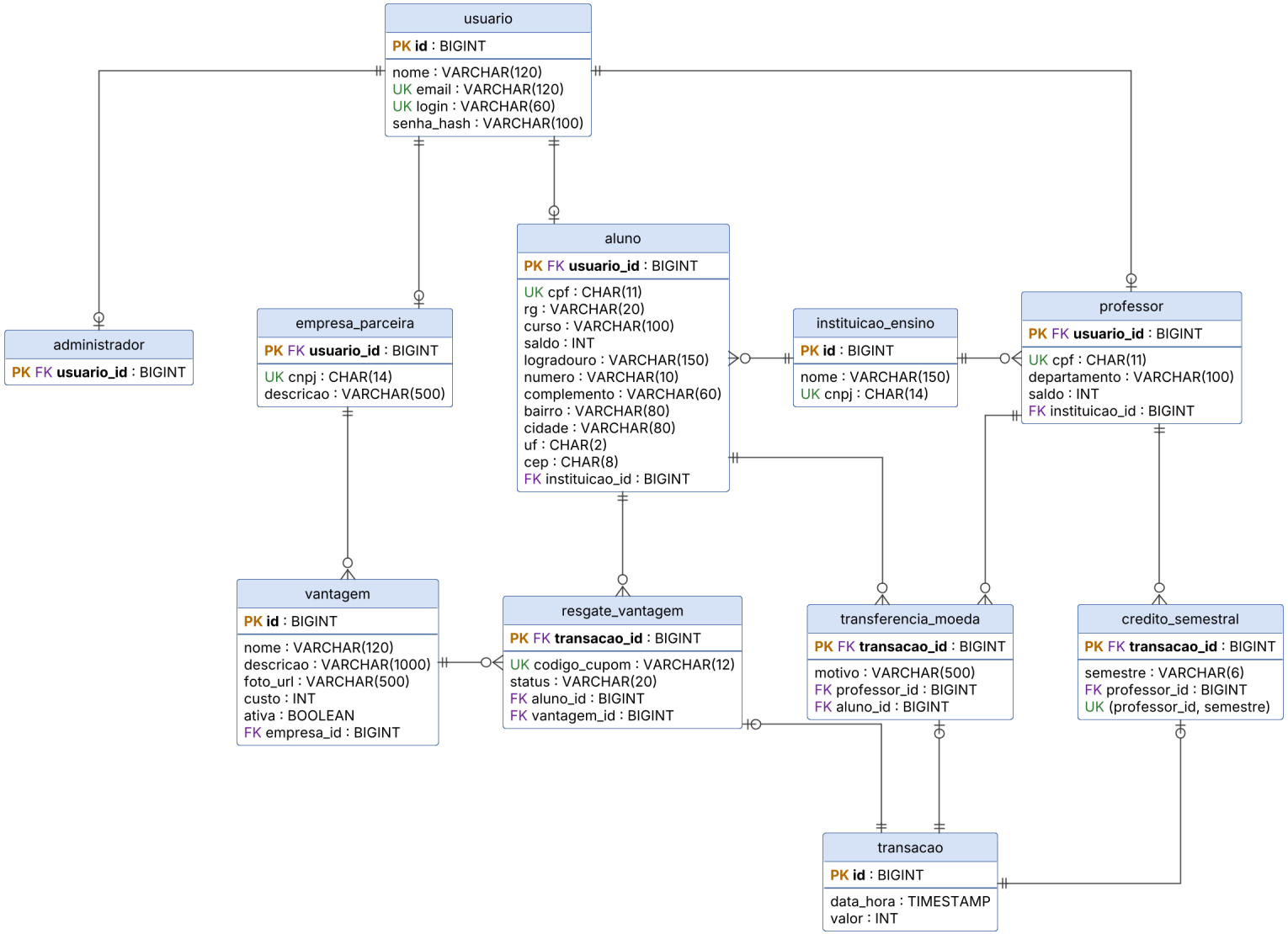


Figura 1 – Modelo ER (notação pé-de-galinha). PK = chave primária, FK = chave estrangeira, UK = valor único.

2.1 Decisões do modelo

- **Herança mapeada como tabela por subclasse (JOINED).** A tabela **usuario** guarda os dados comuns de acesso (nome, email, login, senha) e cada perfil tem sua tabela, cuja PK também é FK para usuario. Assim email e login são únicos entre todos os perfis, e nenhuma coluna específica de um perfil fica nula para os outros.
- **Transações com a mesma estratégia. transacao** guarda data/hora e valor; transferencia_moeda, resgate_vantagem e credito_semestral guardam o que é específico. O extrato vira uma consulta sobre uma única hierarquia.
- **Endereço embutido na tabela aluno** (@Embeddable). A relação é 1:1 e de composição, então uma tabela separada só acrescentaria um JOIN.
- **Professor e aluno referenciam instituicao_ensino** com FK obrigatória, deixando explícito que o professor faz parte de uma instituição, como pede o enunciado.
- **Vantagem pertence à empresa** (composição): ao excluir a empresa, suas vantagens são removidas junto.
- **credito_semestral** tem restrição única (professor_id, semestre), o que impede creditar duas vezes as 1.000 moedas no mesmo semestre.
- **CPF e CNPJ são guardados só com dígitos** (CHAR(11) e CHAR(14)). A máscara é aplicada no front-end.

2.2 Dicionário de dados (tabelas usadas nos CRUDs desta sprint)

Tabela	Coluna	Tipo	Restrições
usuario	id	BIGINT	PK, auto-incremento
	nome / email / login	VARCHAR	NOT NULL; email e login UNIQUE
	senha_hash	VARCHAR(100)	NOT NULL; hash BCrypt
aluno	usuario_id	BIGINT	PK e FK → usuario.id
	cpf	CHAR(11)	NOT NULL, UNIQUE, dígitos verificadores validados
	rg / curso	VARCHAR	NOT NULL
	saldo	INT	NOT NULL, padrão 0
	logradouro … cep	VARCHAR / CHAR	Endereço embutido; complemento opcional
	instituicao_id	BIGINT	FK → instituicao_ensino.id, NOT NULL
empresa_parceira	usuario_id	BIGINT	PK e FK → usuario.id
	cnpj	CHAR(14)	NOT NULL, UNIQUE, dígitos verificadores validados
	descricao	VARCHAR(500)	Opcional
vantagem	id	BIGINT	PK, auto-incremento
	nome / descricao / foto_url	VARCHAR	NOT NULL
	custo	INT	NOT NULL, maior que zero (moedas)
	ativa	BOOLEAN	NOT NULL, padrão verdadeiro
	empresa_id	BIGINT	FK → empresa_parceira.usuario_id
instituicao_ensino	id / nome / cnpj	BIGINT / VARCHAR / CHAR(14)	PK; cnpj UNIQUE; pré-cadastrada

O script completo do modelo físico (PostgreSQL), com todas as tabelas e índices, está em `docs/schema.sql`.

3. Estratégia de acesso ao banco de dados

A persistência combina dois padrões. O **ORM (JPA/Hibernate)** converte objetos em linhas. O **Repository/DAO (Spring Data JPA)** isola esse acesso atrás de interfaces. Controllers nunca falam com o banco; serviços nunca escrevem SQL.

3.1 Fluxo de uma requisição

Passo	Componente	Responsabilidade
1	Front-end (api.js)	Monta o JSON do formulário e chama POST /api/alunos
2	AlunoController	Recebe e valida o AlunoRequest (DTO) com @Valid; devolve 201 Created
3	AlunoService	Abre a transação; valida CPF, unicidade de email/login, senha; busca a instituição
4	AlunoRepository	save(aluno), herdado de JpaRepository; o Hibernate gera os INSERTs em usuario e aluno
5	AlunoService → AlunoResponse	Converte a entidade em DTO de saída, sem expor o hash da senha

3.2 Mapeamento objeto-relacional

Conceito do modelo de classes	Anotação JPA	Resultado no banco
Usuario abstrata e subclasses	@Inheritance(strategy = JOINED)	usuario + tabelas por perfil (PK = FK)
Transacao abstrata e subclasses	@Inheritance(strategy = JOINED)	transacao + 3 tabelas especializadas
Aluno ◆ Endereco (composição 1:1)	@Embeddable / @Embedded	Colunas de endereço na tabela aluno
Aluno/Professor → Instituição	@ManyToOne(optional = false)	FK instituicao_id NOT NULL
EmpresaParceira ◆ Vantagem	@OneToMany(cascade = ALL, orphanRemoval = true)	FK empresa_id; vantagens excluídas com a empresa
StatusResgate (enum)	@Enumerated(EnumType.STRING)	Coluna VARCHAR com o nome do valor
Unicidade (CPF, CNPJ, email…)	@Column(unique = true) + validação no serviço	UNIQUE no banco e mensagem amigável na API

3.3 Repositórios (DAO)

Cada agregado tem uma interface que estende `JpaRepository`. O Spring gera a implementação em tempo de execução, e as consultas são derivadas do nome do método:

```
public interface AlunoRepository extends JpaRepository<Aluno, Long> {
    boolean existsByCpf(String cpf);
    boolean existsByCpfAndIdNot(String cpf, Long id);   // unicidade na edição

    @EntityGraph(attributePaths = "instituicao")         // evita N+1 na listagem
    List<Aluno> findAllByOrderByNomeAsc();
}
```

- **Repositórios implementados:** UsuarioRepository, AlunoRepository, ProfessorRepository, EmpresaParceiraRepository, VantagemRepository, InstituicaoEnsinoRepository e TransacaoRepository.
- **Transações:** cada operação de serviço é @Transactional. Uma falha no meio, como um CPF duplicado, desfaz tudo.
- **DTOs** (records Java) separam o contrato da API das entidades. O hash da senha nunca sai do back-end.

- **Troca de banco sem mudar código:** o padrão é H2 em arquivo; o perfil `postgres` aponta para PostgreSQL.

4. API REST (comunicação front-end ↔ back-end)

Método	Rota	Descrição	Resposta
GET	/api/instituicoes	Instituições pré-cadastradas (select do cadastro)	200
GET	/api/alunos	Lista alunos (ordem alfabética)	200
GET	/api/alunos/{id}	Consulta um aluno	200 / 404
POST	/api/alunos	Cadastra aluno	201 / 400 / 422
PUT	/api/alunos/{id}	Edita aluno (senha vazia mantém a atual)	200 / 400 / 404 / 422
DELETE	/api/alunos/{id}	Exclui aluno	204 / 404 / 409
GET	/api/empresas	Lista empresas com a quantidade de vantagens	200
GET	/api/empresas/{id}	Consulta empresa	200 / 404
POST	/api/empresas	Cadastra empresa	201 / 400 / 422
PUT	/api/empresas/{id}	Edita empresa	200 / 400 / 404 / 422
DELETE	/api/empresas/{id}	Exclui empresa e suas vantagens	204 / 404 / 409
GET	/api/empresas/{id}/vantagens	Vantagens da empresa	200 / 404
POST	/api/empresas/{id}/vantagens	Cadastra vantagem (nome, descrição, foto, custo)	201 / 400
PUT	/api/empresas/{id}/vantagens/{vid}	Edita vantagem / ativa ou desativa	200 / 400 / 404
DELETE	/api/empresas/{id}/vantagens/{vid}	Exclui vantagem	204 / 404 / 409

Erros seguem um formato único, que o front-end usa para destacar o campo com problema:

```
HTTP 422
{ "status": 422, "mensagem": "Já existe um aluno com este CPF.",
  "campos": { "cpf": "Já existe um aluno com este CPF." }, "timestamp": "2026-10-21T19:40:12" }
```

400 = campo obrigatório ou formato inválido (Bean Validation) · 422 = regra de negócio (CPF/CNPJ inválido ou duplicado, email/login em uso, senha curta) · 404 = registro inexistente · 409 = exclusão bloqueada por vínculo.

5. Front-end

As páginas ficam em `src/main/resources/static` e são servidas pelo próprio Spring Boot em http://localhost:8080. O arquivo `api.js` concentra a comunicação com o back-end, as máscaras de CPF/CNPJ/CEP e a exibição dos erros por campo.

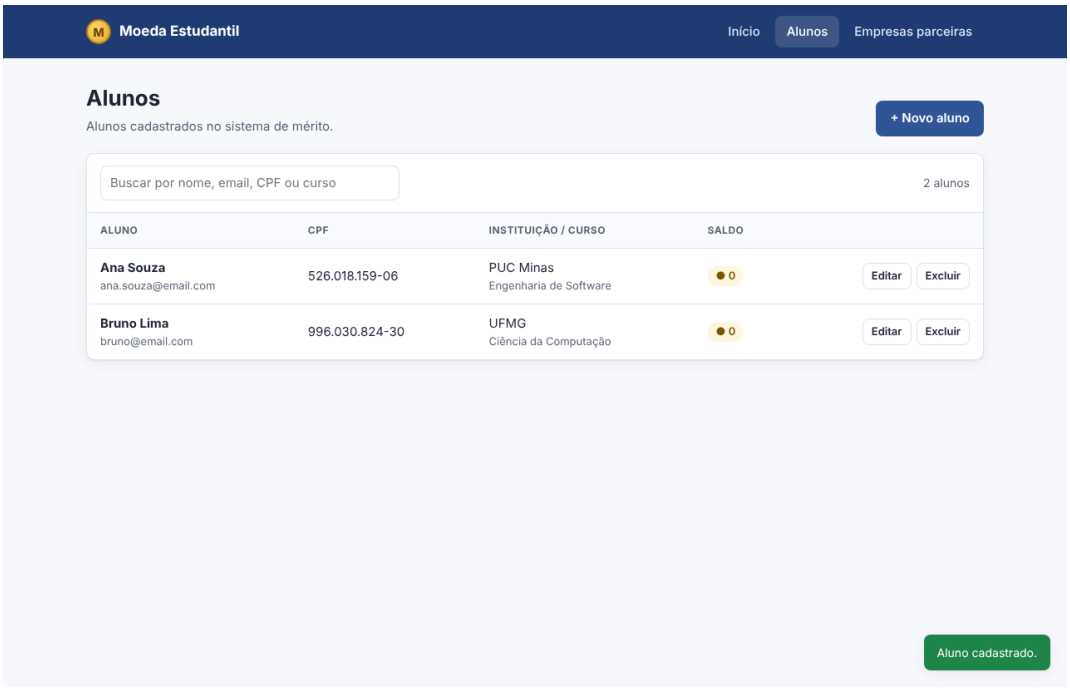


Figura 2 – Listagem de alunos com busca.

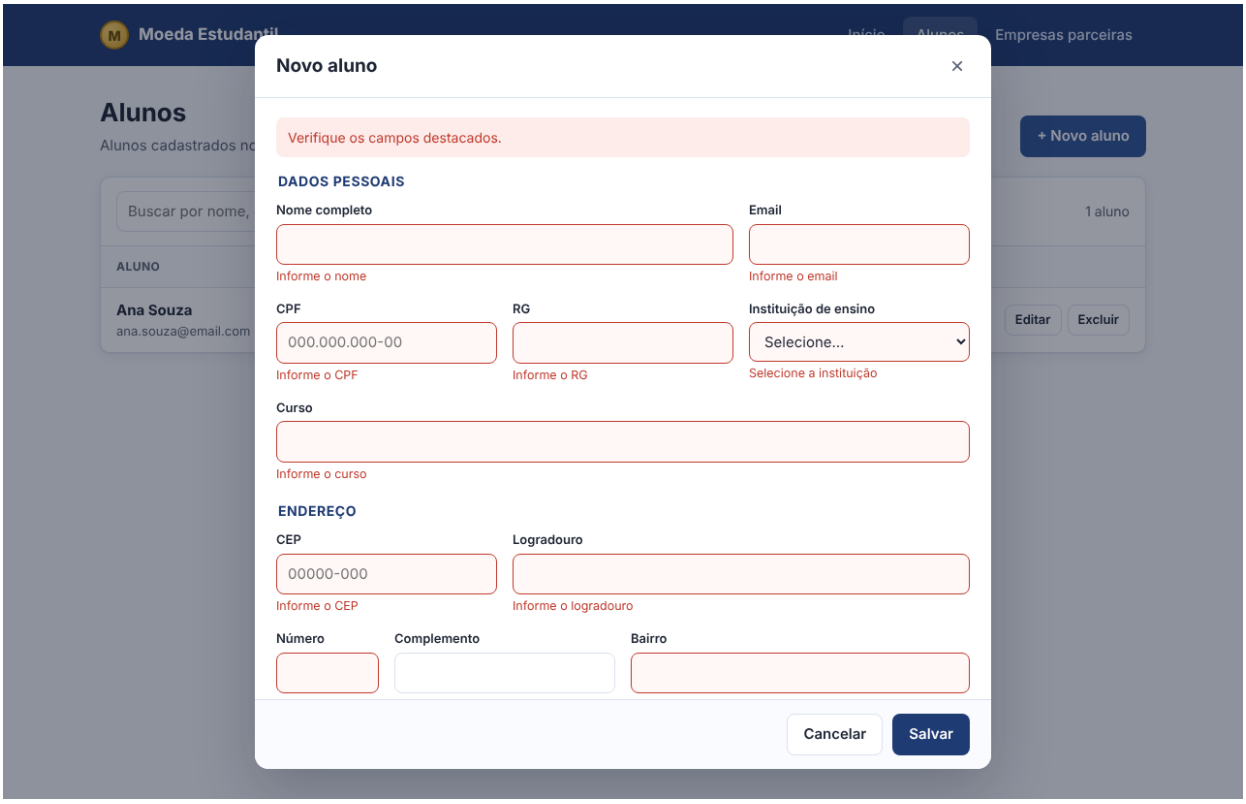


Figura 3 – Cadastro de aluno com os erros de validação devolvidos pela API.

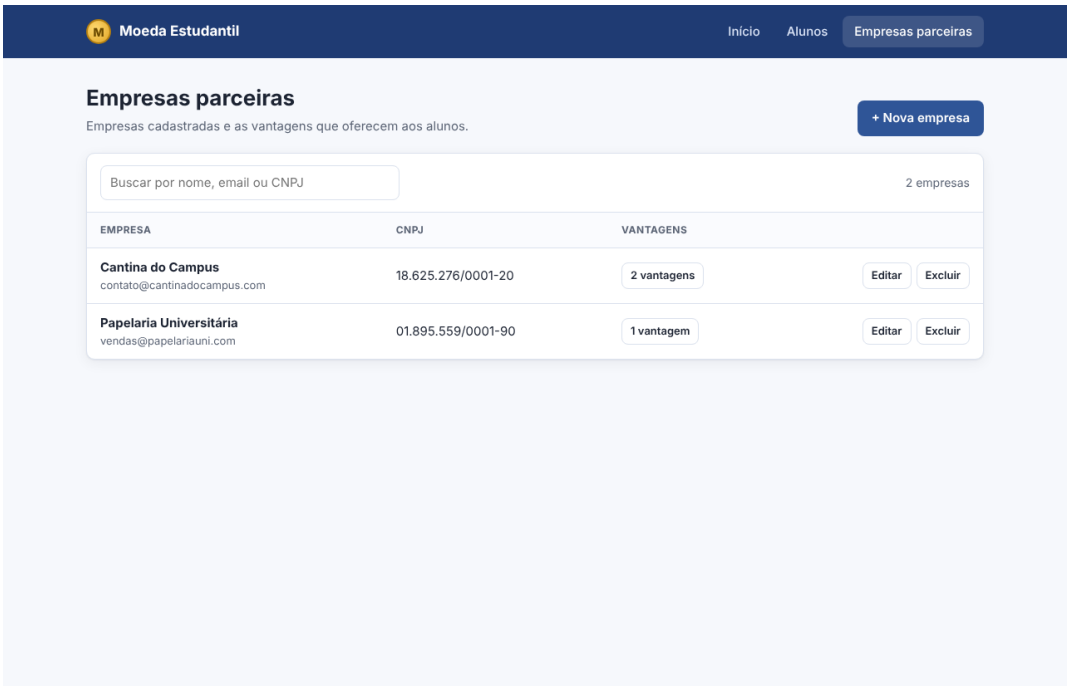


Figura 4 – Listagem de empresas parceiras.

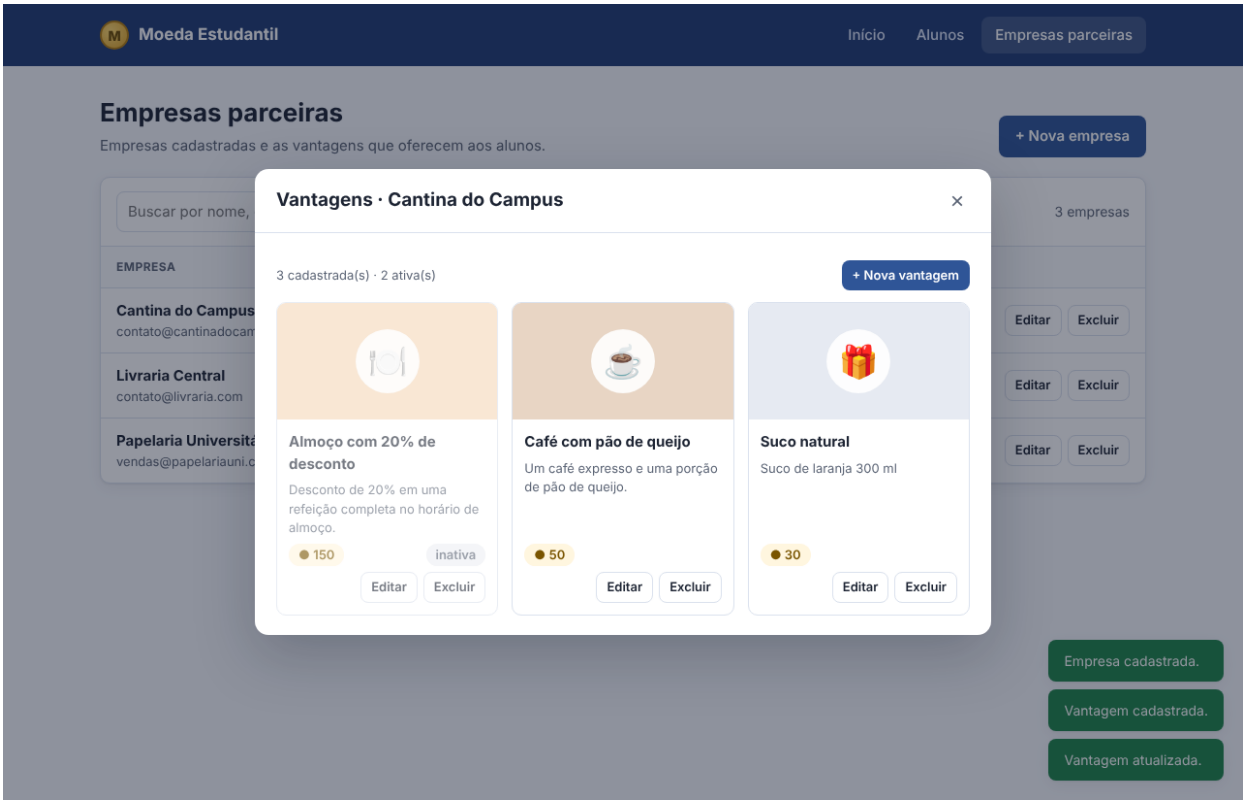


Figura 5 – Vantagens de uma empresa (foto, descrição, custo e situação).

6. Atualização dos modelos UML

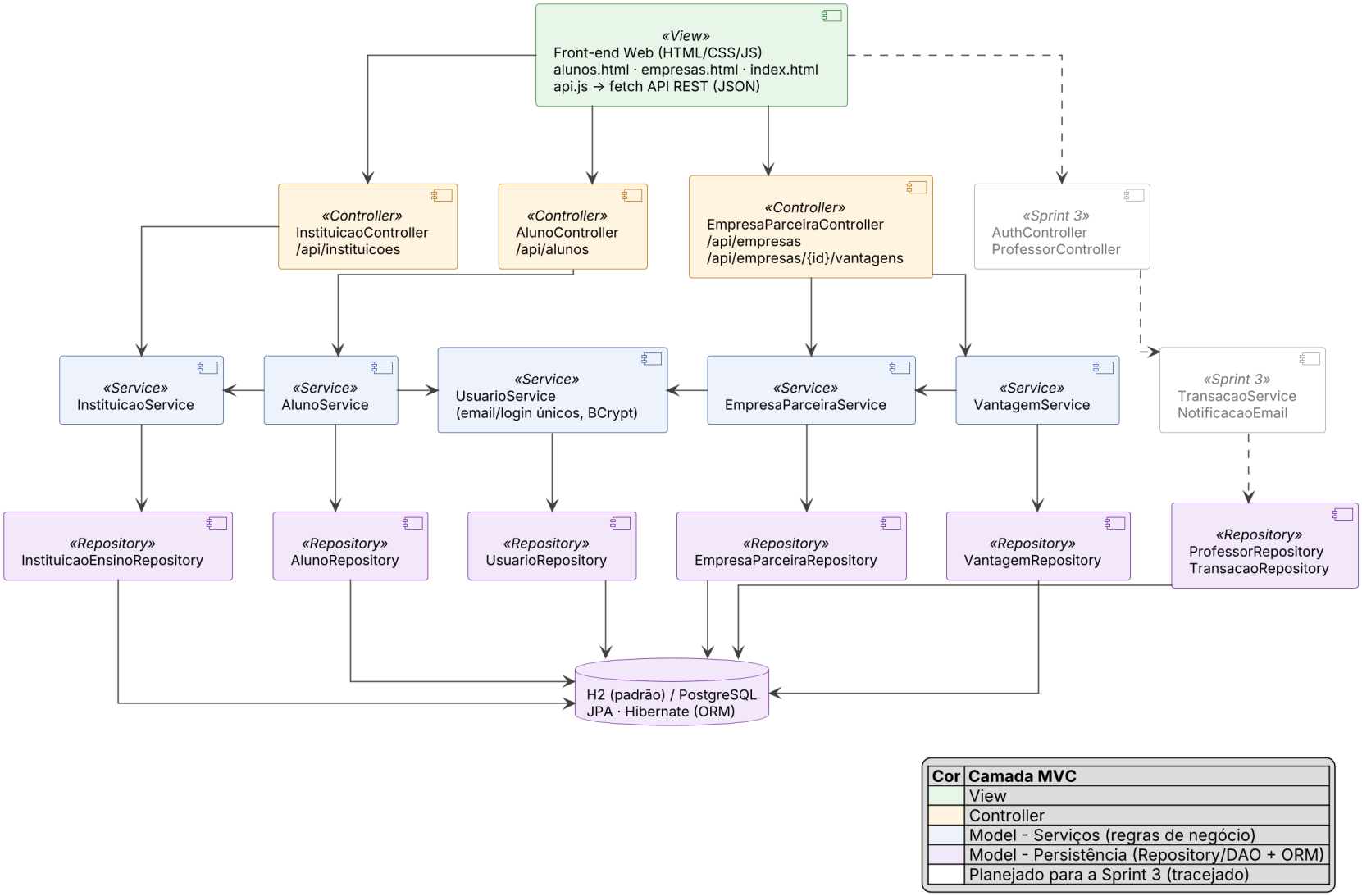


Figura 6 – Diagrama de Componentes v2: componentes implementados e os previstos para a Sprint 3 (tracejados).

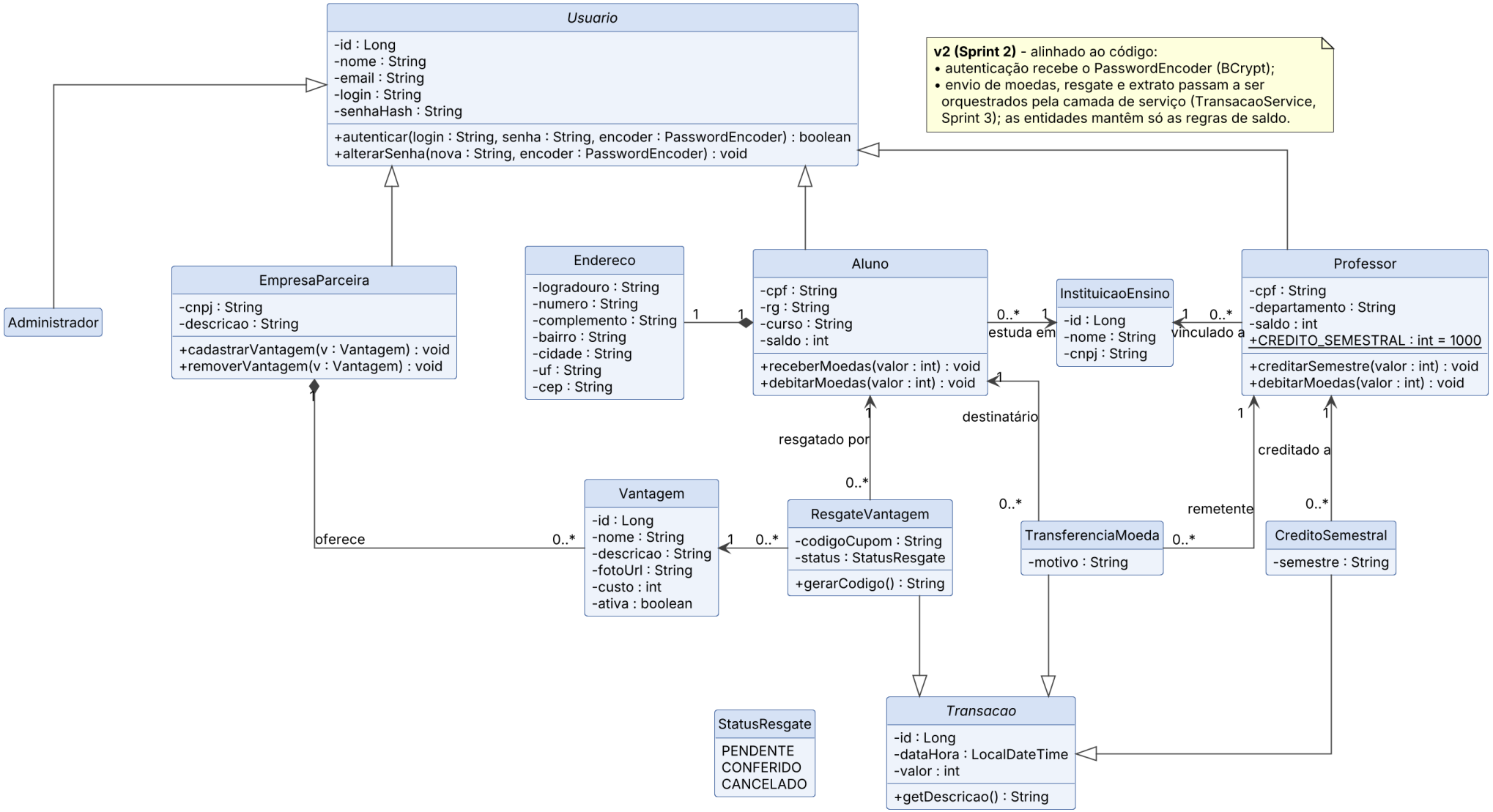


Figura 7 – Diagrama de Classes v2, alinhado às entidades JPA.

6.1 O que mudou em relação à Sprint 1

- **Componentes:** o UsuarioService genérico foi dividido em AlunoService, EmpresaParceiraService e VantagemService, que reaproveitam o UsuarioService para as regras comuns (email, login, senha). Foi incluído o InstituicaoController/Service para alimentar o cadastro de aluno, e cada entidade principal ganhou seu repositório.
- **Classes:** autenticar e alterarSenha recebem o PasswordEncoder (BCrypt). Aluno e Professor ganharam debitarMoedas, e Professor ganhou a constante CREDITO_SEMESTRAL = 1000. Envio de moedas, resgate e extrato passam a ser orquestrados pelo TransacaoService (Sprint 3), e as entidades guardam só as regras de saldo.
- As duas versões de cada diagrama estão em `docs/uml/sprint1` e `docs/uml/sprint2` (fontes PlantUML e imagens), para manter o histórico no GitHub.

7. Como executar

Pré-requisito: Java 17 ou superior. O Maven é baixado automaticamente pelo wrapper.

```
cd moeda-estudantil
./mvnw spring-boot:run          # Windows: mvnw.cmd spring-boot:run
# acesse http://localhost:8080

./mvnw test                     # testes automatizados (H2 em memória)
./mvnw spring-boot:run -Dspring-boot.run.profiles=postgres   # usando PostgreSQL
```

- No primeiro start são carregados: 3 instituições, 2 professores com 1.000 moedas do semestre 2026/2, 1 administrador, 1 aluno e 2 empresas com vantagens. A senha de todos é **123456**.
- Console do banco H2: http://localhost:8080/h2-console (JDBC URL `jdbc:h2:file:./data/moedaestudantil`, usuário `sa`, senha em branco).
- **Próximos passos (Sprint 3):** versão final dos CRUDs com login/autenticação, envio de moedas, resgate de vantagens com cupom e emails, extrato e slides da arquitetura.